

Índice

1	Templates y Testing	1
1.1	Template y archivos	1
1.2	Stress testing (diff)	1
1.3	Stress testing (checker)	1
2	Ideas	1
3	Estructuras básicas	1
3.1	Min queue	1
3.2	Heap actualizable	1
4	Teoría de números	2
4.1	Criba de Eratóstenes	2
4.2	Algoritmo extendido de Euclides	2
4.3	Solución de ecuaciones diofánticas lineales	2
4.4	Funciones multiplicativas	2
4.5	Factorización Pollard Rho	3

TEMPLATES Y TESTING

► Template y archivos

template.h

```
1 #include <bits/stdc++.h>
2 #include <ext/pb_ds/assoc_container.hpp>
3 #include <ext/pb_ds/tree_policy.hpp>
4 using namespace std;
5 using namespace __gnu_pbds;
6 typedef int64_t ll;
7 typedef pair<int, int> pii;
8 typedef pair<ll, ll> pll;
9 typedef vector<int> vi;
10 typedef vector<ll> vll;
11 typedef vector<vi> vvi;
12 typedef vector<vll> vvll;
13 typedef vector<pii> vpii;
14 typedef vector<pll> vpll;
15 typedef tree<int, null_type, less<int>, rb_tree_tag,
    ↳ tree_order_statistics_node_update> ordered_set;
16 #define fi first
17 #define se second
18 #define pb push_back
19 #define all(x) (x).begin(), (x).end()
20 #define sz(x) (int)(x).size()
21 mt19937_64 gen(chrono::steady_clock::now().time_since_epoch().
    ↳ count());
22 uniform_int_distribution<ll> distr(1, LLONG_MAX);
23 const int MOD = 1e9 + 7;
24 template<typename A, typename B> ostream& operator<<(ostream
    ↳ &os, const pair<A, B> &p){return os << '(' << p.fi << ", "
    ↳ << p.se << ')';}
25 template<typename C, typename T = typename
    ↳ enable_if<!is_same<C, string>::value, typename
    ↳ C::value_type>::type>
26 ostream& operator<<(ostream &os, const C &v){string sep;
    ↳ for(const T &x : v) os << sep << x, sep = " "; return os;}
27 #define print(...) logger(#__VA_ARGS__, __VA_ARGS__)
28 template<typename ...Args>
29 void logger(string vars, Args&&... values){
30     cout << "[Debug]\n\t" << vars << " = ";
31     string d = "[";
32     (... , (cout << d << values, d = " ["));
33     cout << "]\n";
34 }
```

Ejecutar en la terminal de carpeta

```
1 for f in {a..n}; do cp template.cpp $f.cpp; done
```

► Stress testing (diff)

Para stress testing en problemas donde existe solo una respuesta, usamos *stress.sh*. Testeamos *code.cpp* contra *brute.cpp*. *gen.cpp* es el generador de casos. Este ultimo recibe de semilla el valor *i*.

stress.sh

```
1 set -e
2 g++ code.cpp -o code
3 g++ gen.cpp -o gen
4 g++ brute.cpp -o brute
5 for((i = 1; ; ++i)); do
6     ./gen $i > input_file
7     ./code < input_file > myAnswer
8     ./brute < input_file > correctAnswer
9     diff -Z myAnswer correctAnswer > /dev/null || break
10    echo "Passed test: " $i
11 done
12 echo "WA on the following test:"
13 cat input_file
14 echo "Your answer is:"
15 cat myAnswer
16 echo "Correct answer is:"
17 cat correctAnswer
```

Para extraer la semilla en *gen.cpp* leemos desde *argv*

```
1 int rnd(int a, int b){return a + rand() % (b - a + 1);}
2 int main(int argc, char* argv[]){
3     int seed = atoi(argv[1]);
4     srand(seed);
5     int n = rnd(1, 5);
6 }
```

► Stress testing (checker)

Usaremos el script similar al anterior *stress.sh*. Aquí el *brute.cpp* es opcional.

```
1 set -e
2 g++ code.cpp -o code
3 g++ gen.cpp -o gen
4 g++ brute.cpp -o brute
5 g++ checker.cpp -o checker
6 for((i = 1; ; ++i)); do
7     ./gen $i > input_file
8     ./code < input_file > myAnswer
9     ./brute < input_file > correctAnswer
10    ↳ checker > checker_log
11    echo "Passed test: " $i
12 done
```

Se ejecutara mientras el return value de checker sea 0. Ejemplo de la implementación de un checker

```
1 bool validAns(ifstream &fin, int n, int* arr){
2     fin>>n;
3     //aquí validariamos nuestra respuesta, fin sera el input
    ↳ stream que le pase
4     if(algunacosa) return true
5     else return false
6 }
7 int main(int argc, char * argv[]){
8     //input stream del caso
9     ifstream fin("input_file", ifstream::in);
10    //input stream de mi respuesta
11    ifstream ans("myAnswer", ifstream::in);
12    //input stream de respuesta bruta
13    ifstream cor("correctAnswer", ifstream::in);
14    //aquí fin puede procesar lo que interese del input original
15    fin>>n;
16    int arr[n];
17    for(int x = 0; x < n; x++) cin>>arr[x];
18    if(!validAns(ans, n, arr)) {
19        cout<<"WA x algum motivo, esto saldrá en cheker_log\n";
20        return -1;
21    }
22    return 0;
23 }
```

IDEAS

Ideas de Catalán. Ideas que se usan en las demostraciones de que algo se cuenta con Catalán y sirven para conteos similares:

1. *Partir en dos conjuntos y distribuir*. Usar la misma idea de la recurrencia de los números de Catalán: $(0, n - 1), (1, n - 2), \dots, (k, n - 1 - k)$.
2. *Reflejar los caminos malos*. Si queremos contar caminos monótonos que $(0, 0)$ y (n, m) , por debajo de una diagonal paralela a la que une dichas esquinas, en una cuadrícula de $n \times m$, contemos todos los caminos $\binom{n+m}{n}$ y restemos los caminos que pasan sobre la diagonal. Digamos que la diagonal es $y = x + k$, entonces los caminos malos pasan al menos una vez por $y = x + k + 1$: toma el primer punto $(x', x' + k + 1)$ sobre el que un camino pasa por $y = x + k + 1$ y refléjalo, nota que todos empiezan en $(x', x' + k + 1) - (x' + k + 1, x') = (-k - 1, k + 1)$.

Entonces los caminos malos son biyectivos a los caminos que van de $(-k - 1, k + 1)$ a (n, m) .

Optimización. Ideas para optimizar funciones.

1. *Fijar coordenadas secuencialmente*. Hay funciones en R^n que se pueden optimizar fijando la primera coordenada y resolviendo para R^{n-1} y así sucesivamente.

ESTRUCTURAS BÁSICAS

► Min queue

min_queue.cpp

```
1 template<typename T> struct min_stack{
2     stack<pair<T, T>> st;
3     min_stack(){}
4     min_stack(const T &MAXVAL){init(MAXVAL);}
5     void init(const T &MAXVAL){st.push({MAXVAL, MAXVAL});}
6     void push(const T &v){st.push({v, min(v, st.top().se)});}
7     T top(){return st.top().fi;}
8     void pop(){if(sz(st) > 1)st.pop();}
9     T minV(){return st.top().se;}
10    int size(){return sz(st) - 1;}
11    bool empty(){return size() == 0;}
12 };
13 template<typename T> struct min_queue{
14     min_queue(const T &MAXVAL){p_in.init(MAXVAL);
    ↳ p_out.init(MAXVAL);}
15     void push(const T &v){p_in.push(v);}
16     T front(){transfer(); return p_out.top();}
17     void pop(){transfer(); p_out.pop();}
18     int size(){return sz(p_in) + sz(p_out);}
19     T minV(){return min(p_in.minV(), p_out.minV());}
20     bool empty(){return size() == 0;}
21     void transfer(){
22         if(sz(p_out)) return;
23         while(sz(p_in)){p_out.push(p_in.top()); p_in.pop();}
24     }
25     min_stack<T> p_in, p_out;
26 };
```

► Heap actualizable

updateable_heap.cpp

```
1 template<class TPriority, class TKey> class UpdatableHeap{
2 public:
3     UpdatableHeap(){
4         TPriority a;
5         TKey b;
6         nodes.clear(); nodes.pb({a, b});
7     }
8     pair<TPriority, TKey> top(){return nodes[1];}
9     void pop(){
10        if(sz(nodes) == 1) return;
11        TKey k = nodes[1].second;
12        swap_nodes(1, sz(nodes) - 1);
13        nodes.pop_back();
14        position.erase(k);
15        heapify(1);
16    }
17    void insert_or_update(const TPriority &p, const TKey &k){
18        int pos;
19        if(is_inserted(k)){
20            pos = position[k];
21            nodes[pos].first += p;
22        } else {
23            position[k] = pos = sz(nodes);
24            nodes.pb({p, k});
25        } heapify(pos);
26    }
27    bool is_inserted(const TKey &k){return position.count(k);}
28    int size(){return sz(nodes) - 1;}
29    void erase(const TKey &k){
30        if(!is_inserted(k)) return;
31        int pos = position[k];
32        swap_nodes(pos, sz(nodes) - 1);
```

```

33     nodes.pop_back();
34     position.erase(k);
35     heapify(pos);
36 }
37 private:
38     vector<pair<TPriority, TKey>> nodes;
39     map<TKey, int> position;
40     void heapify(int pos){
41         if(pos >= sz(nodes)) return;
42         while(1 < pos && nodes[pos / 2] <= nodes[pos]){
43             swap_nodes(pos / 2, pos);
44             pos /= 2;
45         }
46         int l = pos * 2, r = pos * 2 + 1, maxi = pos;
47         if(1 < sz(nodes) && nodes[l] > nodes[maxi]) maxi = l;
48         if(r < sz(nodes) && nodes[r] > nodes[maxi]) maxi = r;
49         if(maxi != pos){
50             swap_nodes(pos, maxi);
51             heapify(maxi);
52         }
53     }
54     void swap_nodes(int a, int b){
55         position[nodes[a].se] = b;
56         position[nodes[b].se] = a;
57         swap(nodes[a], nodes[b]);
58     }
59 };

```

TEORÍA DE NÚMEROS

► Criba de Eratóstenes

Criba Complejidad: Tiempo $O(n \log \log n)$ - Memoria extra $O(n)$.
Calcula los primos menores o iguales a n .

sieve.cpp

```

1 void sieve(int n, vi &primes){
2     primes.clear(); if(n < 2) return;
3     vector<bool> nprime(n + 1);
4     nprime[0] = nprime[1] = 1;
5     for(ll i = 3; i * i <= n; i += 2) if(!nprime[i])
6         for(ll j = i * i; j <= n; j += 2 * i) nprime[j] = 1;
7     primes.pb(2);
8     for(int i = 3; i <= n; i += 2) if(!nprime[i]) primes.pb(i);
9 }

```

Criba sobre un rango. Complejidad: Tiempo $O(\sqrt{b} \log \log \sqrt{b} + (b - a) \log \log (b - a))$ - Memoria extra $O(\sqrt{b} + b - a)$. Calcula los primos en el intervalo $[a, b]$.

range_sieve.cpp

```

1 void sieve(int n, vi &primes);
2 void range_sieve(ll a, ll b, vll &primes){
3     a = max(a, 0ll);
4     b = max(b, 0ll);
5     ll len = b - a + 1;
6     vi sqrt_primes;
7     sieve(sqrt(b) + 1, sqrt_primes);
8     vector<char> nprime(len);
9     primes.clear();
10    for(ll p : sqrt_primes){
11        ll ini = p * max(p, (a + p - 1) / p);
12        for(ll m = ini; m <= b; m += p) nprime[m - a] = 1;
13    }
14    for(ll i = 0; i < len; ++i)
15        if(!nprime[i] || i + a < 2) primes.pb(i + a);
16 }

```

Criba segmentada. Complejidad: Tiempo $O(\sqrt{n} \log \log \sqrt{n} + n \log \log n)$ - Memoria extra $O(\sqrt{n} + S)$. Cuenta la cantidad de primos menores o iguales a n .

```

1 int cuenta_primos(int n){
2     if(n < 2) return 0;
3     const int S = sqrt(n);
4     vi primos_raiz;

```

```

5     criba(sqrt(n) + 1, primos_raiz);
6     int ans = 0;
7     bool no_primo[S + 1] = {};
8     for(int ini = 0; ini <= n; ini += S){
9         memset(no_primo, 0, S + 1);
10        for(int p : primos_raiz){
11            int m = p * max(p, (ini + p - 1) / p) - ini;
12            for(; m <= S; m += p) no_primo[m] = 1;
13        }
14        for(int i = 0; i < S && i + ini <= n; ++i)
15            if(!no_primo[i] && 1 < i + ini) ans++;
16        } return ans;
17 }

```

Criba lineal. Complejidad: Tiempo $O(n)$ - Memoria extra $O(n)$.
Calcula los primos menores o iguales a n y el menor primo que divide a cada entero en $[2, n]$. ADVERTENCIA: es $O(n)$ pero tiene una constante grande.

```

1 void criba_lineal(int n, vi &primos){
2     if(n < 2) return;
3     vi lp(n + 1);
4     for(ll i = 2; i <= n; ++i){
5         if(!lp[i]) primos.pb(lp[i] = i);
6         for(int j = 0; i * primos[j] <= n; ++j){
7             lp[i * primos[j]] = primos[j];
8             if(primos[j] == lp[i]) break;
9         }
10    }
11 }

```

► Algoritmo extendido de Euclides

Complejidad: Tiempo $O(\log(\max(a, b)))$ - Memoria extra $O(1)$.
Encuentra una solución a la ecuación $ax + by = \gcd(a, b)$.

```

1 int gcd_ext(int a, int b, int &x, int &y){
2     if(!b){ x = 1; y = 0; return a; }
3     int x1, y1, g = gcd_ext(b, a % b, x1, y1);
4     x = y1;
5     y = x1 - y1 * (a / b);
6     return g;
7 }

```

► Solución de ecuaciones diofánticas lineales

Complejidad: Tiempo $O(\log(\max(a, b)))$ - Memoria extra $O(1)$.
Encuentra una solución a la ecuación $ax + by = c$ o determina si no existe solución.

```

1 bool encuentra_solucion(int a, int b, int c, int &x, int &y,
2     int &g){
3     g = gcd_ext(abs(a), abs(b), x, y);
4     if(c % g) return false;
5     x *= c / g;
6     y *= c / g;
7     if(a < 0) x = -x;
8     if(b < 0) y = -y;
9     return true;

```

Cambia a la siguiente (anterior) solución $|cnt|$ veces. $g := \gcd(a, b)$.

```

1 void cambia_solucion(int &x, int &y, int a, int b, int cnt,
2     int g = 1) {
3     x += cnt * b / g;
4     y -= cnt * a / g;

```

Cuenta la cantidad de soluciones x, y con $x \in [minx, maxx]$ y $y \in [miny, maxy]$.

```

1 int cuenta_soluciones(int a, int b, int c, int minx, int maxx,
2     int miny, int maxy) {
3     int x, y, g;
4     if(!encuentra_solucion(a, b, c, x, y, g)) return 0;

```

```

4     // ax + by = c ssi (a/g)x + (b/g)y = c/g
5     // Dividimos entre g para simplificar y no dividir a cada
6     // rato
7     a /= g;
8     b /= g;
9     // Signos de a, b nos sirven para pasar a la
10    // siguiente (anterior) solucion
11    int sign_a = a > 0 ? +1 : -1;
12    int sign_b = b > 0 ? +1 : -1;
13    // pasa a la minima solucion tal que minx <= x
14    cambia_solucion(x, y, a, b, (minx - x) / b);
15    // si x < minx, pasa a la siguiente para que minx <= x
16    if(x < minx) cambia_solucion(x, y, a, b, sign_b);
17    if(x > maxx) return 0; // si x > maxx, entonces no hay x
18    // solucion tal que x in [minx, maxx]
19    int lx1 = x;
20    // pasa a la maxima solucion tal que x <= maxx
21    cambia_solucion(x, y, a, b, (maxx - x) / b);
22    if(x > maxx) cambia_solucion(x, y, a, b, -sign_b); // si x
23    // > maxx, pasa a la solucion anterior
24    int rx1 = x;
25    // hace todo lo anterior pero con y
26    cambia_solucion(x, y, a, b, -(miny - y) / a);
27    if(y < miny) cambia_solucion(x, y, a, b, -sign_a);
28    if(y > maxy) return 0;
29    int lx2 = x;
30    cambia_solucion(x, y, a, b, -(maxy - y) / a);
31    if(y > maxy) cambia_solucion(x, y, a, b, sign_a);
32    int rx2 = x;
33    // como al encontrar las x tomando y como criterio no nos
34    // asegura
35    // que esten ordenadas, entonces las ordenamos
36    if(lx2 > rx2) swap(lx2, rx2);
37    // obtenemos la interseccion de los intervalos
38    int lx = max(lx1, lx2);
39    int rx = min(rx1, rx2);
40    if(lx > rx) return 0; // no existen soluciones,
41    // interseccion vacia
42    // las soluciones (por x) van de b en b (b/g en b/g pero
43    // dividimos al principio)
44    return (rx - lx) / abs(b) + 1;

```

► Funciones multiplicativas

Función Phi de Euler. Complejidad: Tiempo $O(d)$ - Memoria extra $O(n)$. d es la cantidad de factores primos de n . Cuenta la cantidad de coprimos con n menores a n .

```

1 int phi(int n){
2     if(n <= 1) return 1;
3     if(!dp[n]){
4         int pot = 1, p = lp[n], n0 = n;
5         while(n0 % p == 0){ pot *= p; n0 /= p; }
6         dp[n] = (pot / p) * (p - 1) * phi(n0);
7     } return dp[n];
8 }

```

Función σ_0 . Complejidad: Tiempo $O(d)$ - Memoria extra $O(n)$. d es la cantidad de factores primos de n . Cuenta la cantidad de divisores de n .

```

1 ll sigma0(int n){
2     if(n <= 1) return 1;
3     if(!dp[n]){
4         ll exp = 0, p = lp[n], n0 = n;
5         while(n0 % p == 0){ exp++; n0 /= p; }
6         dp[n] = (exp + 1) * sigma0(n0);
7     } return dp[n];
8 }

```

Función σ_1 . Complejidad: Tiempo $O(d)$ - Memoria extra $O(n)$. d es la cantidad de factores primos de n . Calcula la suma de los divisores de n .

```

1 ll sigma1(int n){
2     if(n <= 1) return 1;
3     if(!dp[n]){
4         ll pot = 1, p = lp[n], n0 = n;
5         while(n0 % p == 0){ pot *= p; n0 /= p; }
6         dp[n] = (pot*p - 1) / (p-1) * sigma1(n0);
7     } return dp[n];
8 }

```

Función de Moebius. Complejidad: Tiempo $O(d)$ - Memoria extra $O(n)$. d es la cantidad de factores primos de n . Devuelve 0 si n no es divisible por algún cuadrado. Devuelve 1 o -1 si n es divisible por al menos un cuadrado. Devuelve 1 si n tiene una cantidad par de factores primos. Devuelve -1 si n tiene una cantidad impar de factores primos.

```

1 int moebius(int n){
2     if(n <= 1) return 1;
3     if(dp[n] == -7){
4         int exp = 0, p = lp[n], n0 = n;
5         while(n0 % p == 0){ exp++; n0 /= p; }
6         dp[n] = (exp > 1 ? 0 : -1 * moebius(n0));
7     } return dp[n];
8 }

```

► Factorización Pollard Rho

Complejidad: $O(\sqrt[4]{n})$. Inicializar con PollardRho::init() y para factorizar un número PollardRho::factorize(n).

```

1 //COMPIADO Y PEGADOPORCUESTIONES DE TIEMPO
2 namespace PollardRho {
3     mt19937 rnd(chrono::steady_clock::now()
4         .time_since_epoch().count());
5     const int P = 1e6 + 9;
6     ll seq[P];
7     int primes[P], spf[P];
8     inline ll add_mod(ll x, ll y, ll m){
9         return (x += y) < m ? x : x - m; }
10    inline ll mul_mod(ll x, ll y, ll m) {
11        ll res = __int128(x) * y % m;
12        return res;
13        // ll res = x * y - (ll)((long double)x * y / m + 0.5) *
14        // ↪ m;
15        // return res < 0 ? res + m : res;
16    }
17    inline ll pow_mod(ll x, ll n, ll m) {
18        ll res = 1 % m;
19        for (; n; n >>= 1) {
20            if (n & 1) res = mul_mod(res, x, m);
21            x = mul_mod(x, x, m);
22        }
23        return res;
24    }
25    // 0(it * (logn)^3), it = number of rounds performed
26    inline bool miller_rabin(ll n) {
27        if (n<=2 || (n & 1 ^ 1)) return (n==2);
28        if (n < P) return spf[n] == n;
29        ll c, d, s = 0, r = n - 1;
30        for (; !(r & 1); r >>= 1, s++) {}
31        // each iteration is a round
32        for(int i=0; primes[i]<n && primes[i]<32; i++){
33            c = pow_mod(primes[i], r, n);
34            for(int j = 0; j < s; j++){
35                d = mul_mod(c, c, n);
36                if(d==1&&c!=1&&c!=n-1) return 0;
37                c = d;
38            } if (c!=1) return 0;
39        } return 1;
40    }
41    void init() {
42        int cnt = 0;
43        for(int i = 2; i < P; i++){
44            if(!spf[i]) primes[cnt++] = spf[i]=i;
45            for(int j=0,k;(k=i*primes[j])<P;j++){
46                spf[k] = primes[j];
47                if (spf[i] == spf[k]) break;

```

```

47        }
48    }
49    }
50    // returns 0(n^(1/4))
51    ll pollard_rho(ll n) {
52        while(1){
53            ll x=rnd()%n,y=x,c=rnd()%n,u=1,v,t=0;
54            ll *px = seq, *py = seq;
55            while(1){
56                *py++ = y = add_mod(mul_mod(y, y, n), c, n);
57                *py++ = y = add_mod(mul_mod(y, y, n), c, n);
58                if((x = *px++) == y) break;
59                v = u;
60                u = mul_mod(u, abs(y - x), n);
61                if(!u) return __gcd(v, n);
62                if(++t == 32){
63                    t = 0;
64                    if((__gcd(u,n))>1 && u<n)
65                        return u;
66                }
67            }
68            if(t && (u = __gcd(u, n)) > 1 && u<n)
69                return u;
70        }
71    }
72    vector<ll> factorize(ll n) {
73        if(n == 1) return vector<ll>();
74        if(miller_rabin(n)) return vector<ll>{n};
75        vector<ll> v, w;
76        while(n > 1 && n < P){
77            v.pb(spf[n]); n /= spf[n];
78        }
79        if(n >= P){
80            ll x = pollard_rho(n);
81            v = factorize(x);
82            w = factorize(n / x);
83            v.insert(v.end(), all(w));
84        } return v;
85    }
86 }

```